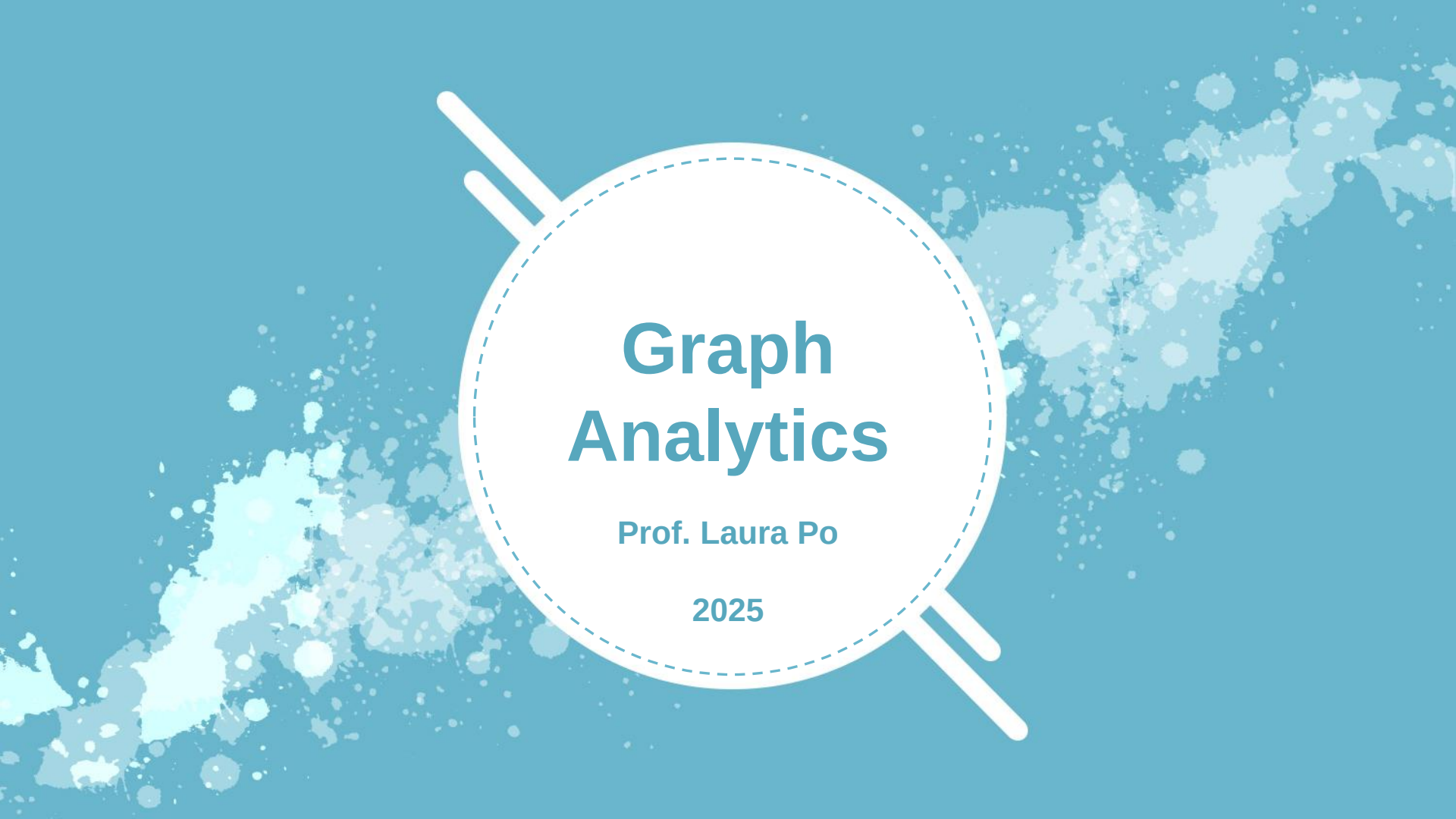




Graph Analytics

Prof. Laura Po

2025



Graph Storage

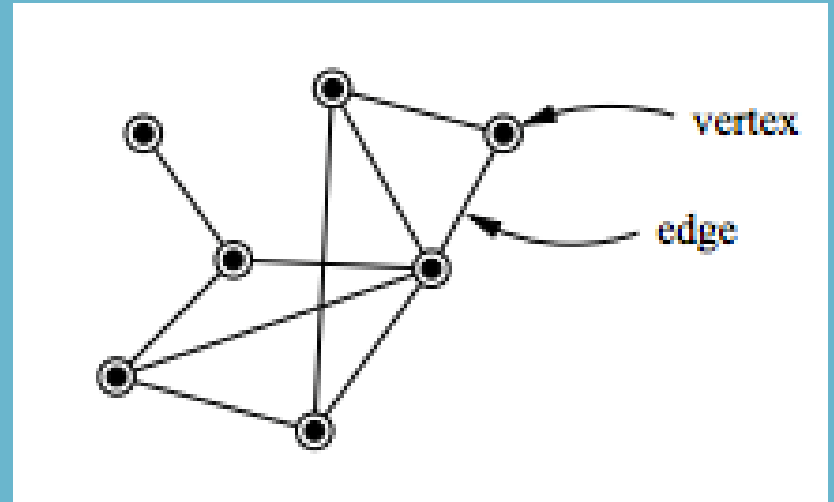
OBJECTIVES

- Provide ways in which a graph can be represented on a computer
- Understand the distinction between dense and sparse graphs and how it affects algorithm efficiency

WHAT IS A GRAPH?

Graph consists of:

- Vertices or nodes
- Edges connecting pairs of nodes



How to represent a Graph on a computer

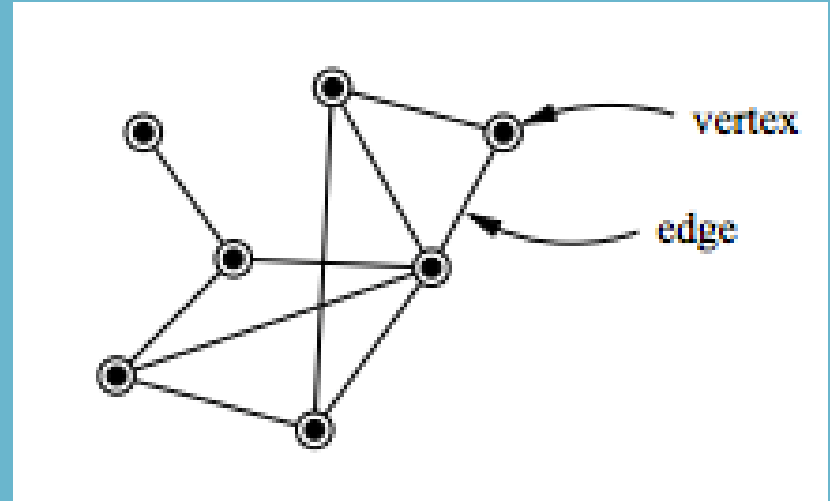


WHAT IS A GRAPH?

COMPUTER SCIENCE DEFINITION

A Graph is an abstract data type:

- Has a data structure to represent the mathematical graph
- Support a number of operations (on that graph)
 - Add vertex
 - Add edges
 - Find the nearest neighbors



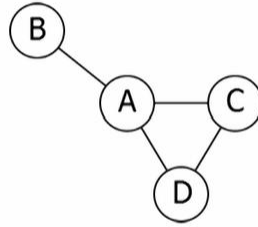
The background is a solid teal color. It features several white, irregular splatters or paint-like marks scattered across the surface. There are also several thin, light blue diagonal lines running across the background, some parallel and some intersecting.

Graph in Computer Science

Edge List

store a giant list of edges

List of all edges:



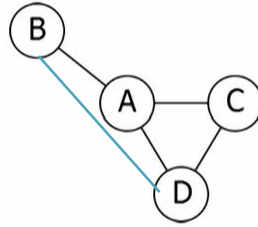
Edges: (A, B) , (A, C) , (A, D) , (C, D)

BASIC OPERATIONS:
How can I add an edge?

Edge List

store a giant list of edges

List of all edges:

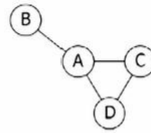


Edges: (A, B) , (A, C) , (A, D) , (C, D) , (B, D)

Adjacency Matrix

- An adjacency matrix is a square matrix used to represent a finite graph. The elements of the matrix indicate whether pairs of vertices are adjacent or not in the graph.

Matrix. Entries 1 if there is an edge, 0 if there is not.



	A	B	C	D
A	0	1	1	1
B	1	0	0	0
C	1	0	0	1
D	1	0	1	0

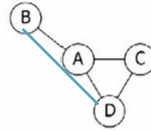
	A	B	C	D
A	F	T	F	F
B	T	F	F	F
C	F	T	F	T
D	F	F	F	F

BASIC OPERATIONS:
How can I add an edge?

Adjacency Matrix

- An adjacency matrix is a square matrix used to represent a finite graph. The elements of the matrix indicate whether pairs of vertices are adjacent or not in the graph.

Matrix. Entries 1 if there is an edge, 0 if there is not.

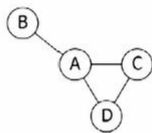


	A	B	C	D
A	0	1	1	1
B	1	0	0	1
C	1	0	0	1
D	1	1	1	0

	A	B	C	D
A	F	T	F	F
B	T	F	F	T
C	F	T	F	T
D	F	T	F	F

Adjacency Matrix

Matrix. Entries 1 if there is an edge, 0 if there is not.



	A	B	C	D
A	0	1	1	1
B	1	0	0	0
C	1	0	0	1
D	1	0	1	0

In the special case of a finite **simple graph**, the adjacency matrix is a (0,1)-matrix with zeros on its diagonal.

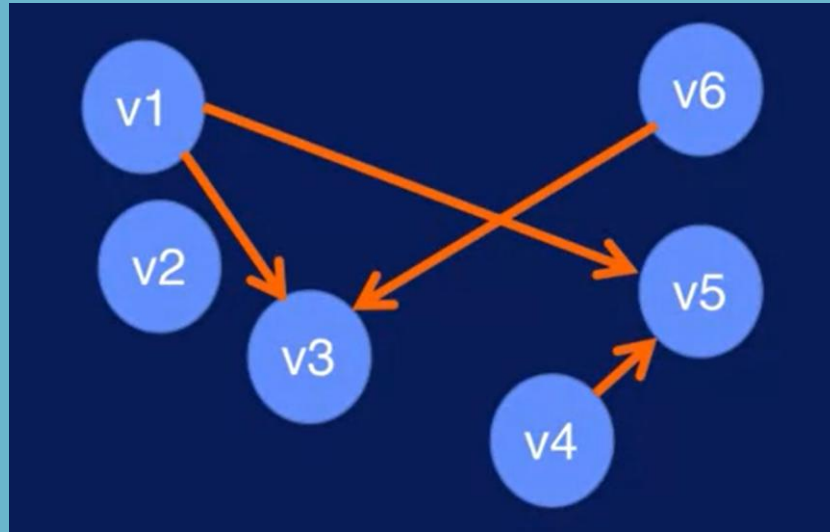
If the graph is **undirected** (i.e. all of its edges are bidirectional), the adjacency matrix is **symmetric**.

If the graph is a **weighted graph**, put the weight instead of 0/1 values in the matrix.

The relationship between a graph and the eigenvalues and eigenvectors of its adjacency matrix is studied in spectral graph theory.

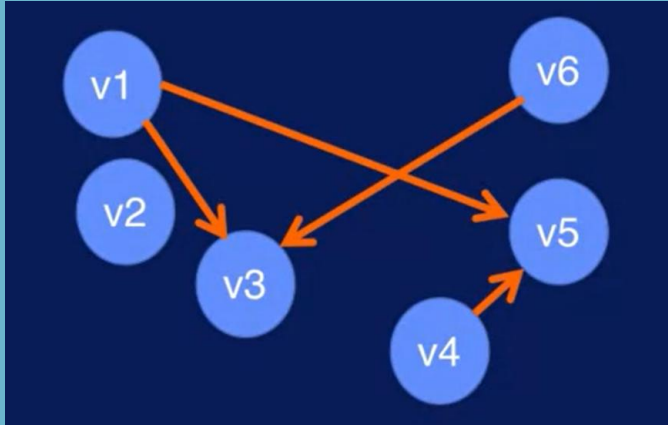
Example

CREATE THE
Adjacency Matrix



Example

CREATE THE
Adjacency Matrix

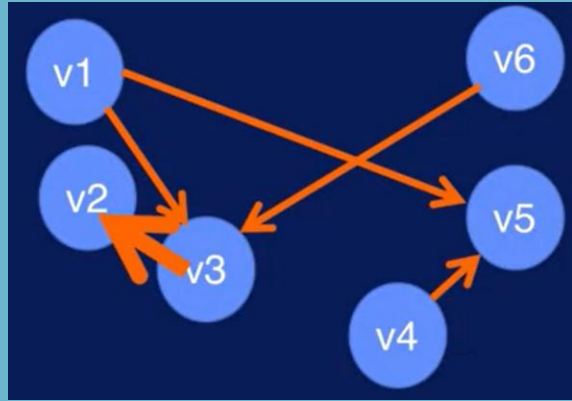


		To					
		v1	v2	v3	v4	v5	v6
From	v1			1		1	
	v2						
	v3						
	v4					1	
	v5						
	v6			1			

Next: ADD AN EDGE
(V3,V2)

Example

ADD AN EDGE
(V3,V2)

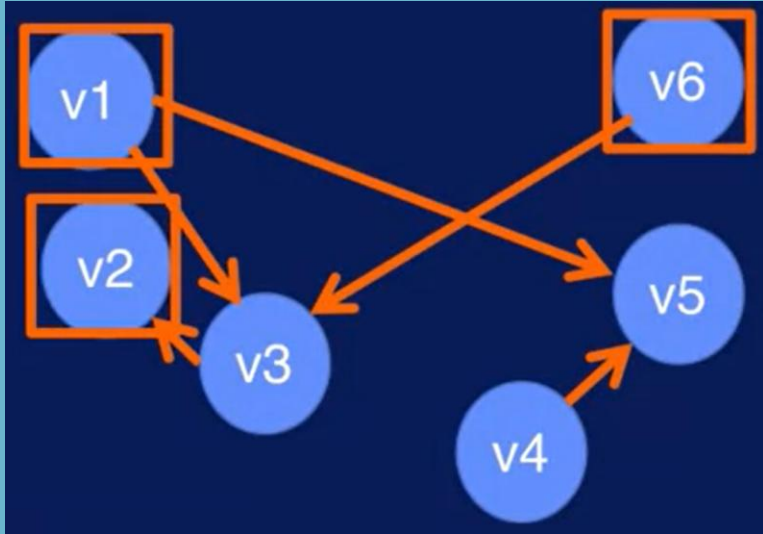


		To					
From		v1	v2	v3	v4	v5	v6
	v1			1		1	
	v2						
	v3		1				
	v4					1	
	v5						
	v6			1			

EXT: GET THE NEIGHBORS OF V3

Example

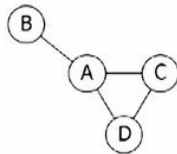
GET THE NEIGHBORS OF V3



		To					
From		v1	v2	v3	v4	v5	v6
	v1			1		1	
	v2						
	v3		1				
	v4					1	
	v5						
	v6			1			

Adjacency List

For each vertex, a list of adjacent vertices.



A adjacent to *B*, *C*, *D*

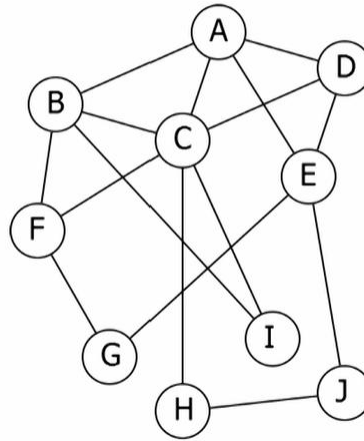
B adjacent to *A*

C adjacent to *A*, *D*

D adjacent to *A*, *C*

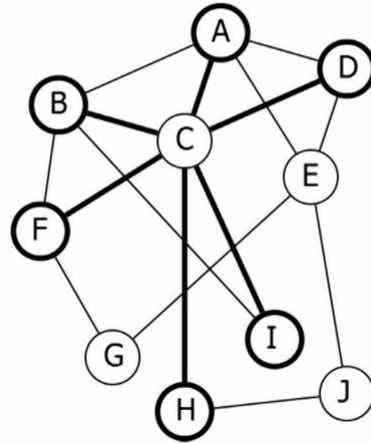
Problem

What are the neighbors of C ?



Solution

A, B, D, F, H, I.



Cost of simple operations

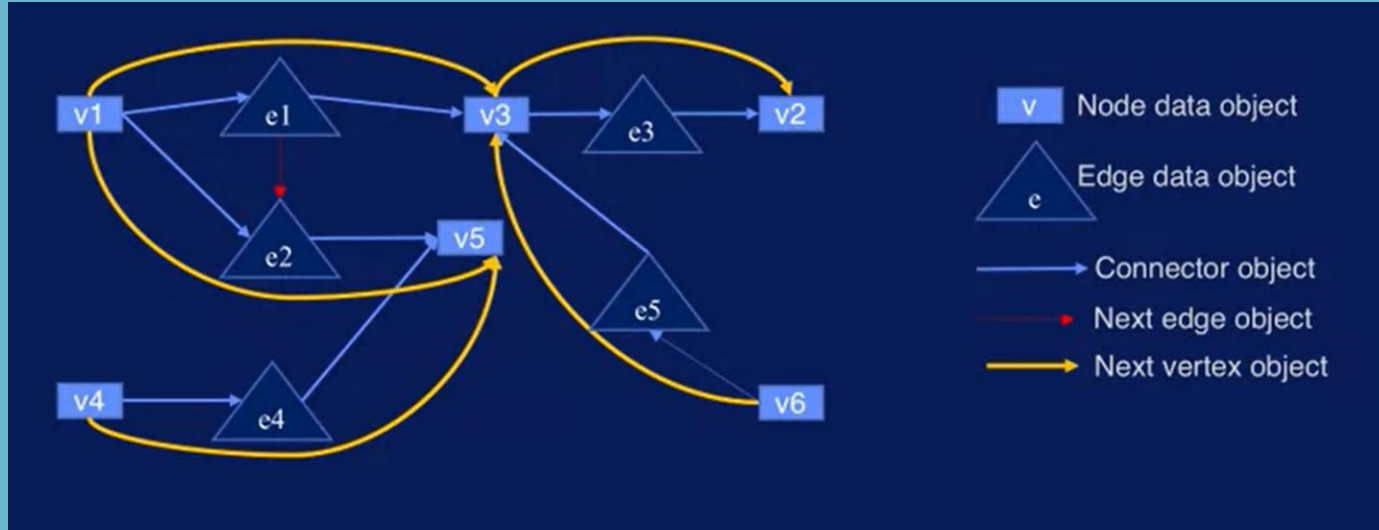
Different operations are faster in different representations.

- deg= degree
- of the vertex

Op.	Is Edge?	List Edge	List Nbrs.
Adj. Matrix	$\Theta(1)$	$\Theta(V ^2)$	$\Theta(V)$
Edge List	$\Theta(E)$	$\Theta(E)$	$\Theta(E)$
Adj. List	$\Theta(\text{deg})$	$\Theta(E)$	$\Theta(\text{deg})$

For many problems, want adjacency list.

Other graph representation



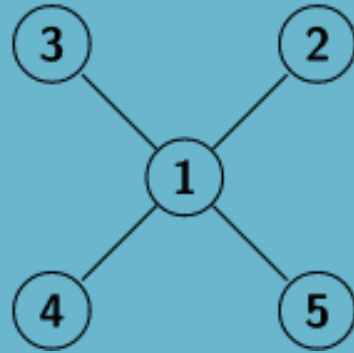
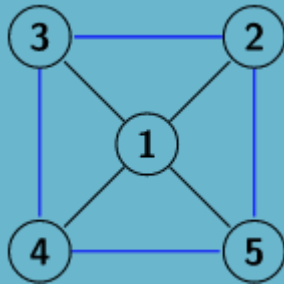


Graph density

General Idea of Graph Density

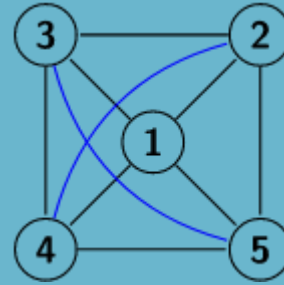
We address the concept of graph density in relation to the graph's size and order.

More «denser» →



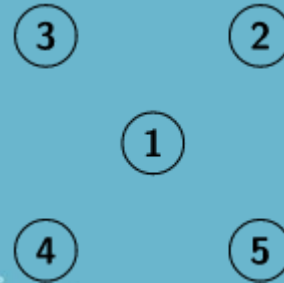
General Idea of Graph Density

We two extreme cases exist, in which either the graph is maximally dense →



Or, on the contrary, minimally dense →

A density is some metric that tells us how “full” a graph is, in terms of the number of edges that it possesses, and in relation to the number of its vertices.



Density

The density of a graph $G(V, E)$ has the form of $D(V, E)$.

This measure should vary between $D(V, E) = 0$, which describes perfectly sparse graphs, and $D(V, E) = 1$, which describes perfectly dense graphs (a complete graph).

For an **undirected graph** D is:

$$D_U(V, E) = \frac{|E|}{\text{Max}_U(V)} = \frac{|E|}{\frac{|V| \cdot (|V| - 1)}{2}} = \frac{2 \cdot |E|}{|V| \cdot (|V| - 1)}$$

Max(V) - maximum number of edges

The maximum number of edges depends on the order but not on the size of the graph.

We can define the maximum number of edges in relation to the order $|V|$ of an undirected graph, as:

$$\text{Max}_U(V) = \binom{|V|}{2} = \frac{|V| \cdot (|V| - 1)}{2}$$

We can extend this formula to directed graphs.

$$\text{Max}_D(V) = 2 \cdot \binom{|V|}{2} = |V| \cdot (|V| - 1) = 2 \cdot \text{Max}_U(V)$$

Density

If the graph is directed, then the maximum number of edges is twice the one we calculated for the corresponding undirected graph.

As a consequence, for **directed graphs**, we can calculate their density as half that of the corresponding undirected graph

$$D_D(V, E) = \frac{|E|}{\text{Max}_D(V)} = \frac{|E|}{|V| \cdot (|V| - 1)} = \frac{1}{2} \cdot D_U(V, E)$$

Sparse Versus Dense Graphs

The **sparse graph** is a graph whose density D is in the lower range of the density's codomain, or $0 \leq D < \frac{1}{2}$.

Analogously, a **dense graph** is a graph whose density D is in the higher range of its codomain, or $\frac{1}{2} < D \leq 1$.

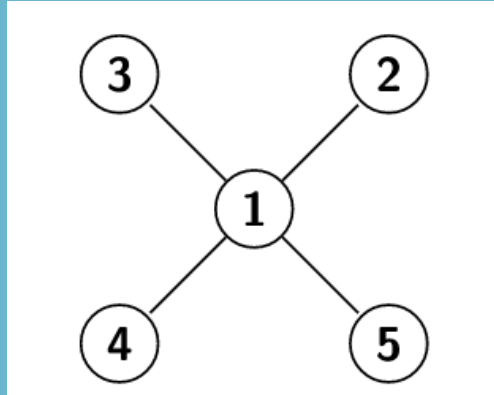
The graph for which $D = \frac{1}{2}$ are considered neither.

General characteristics

General characteristics of the density of a graph, in relation to its type:

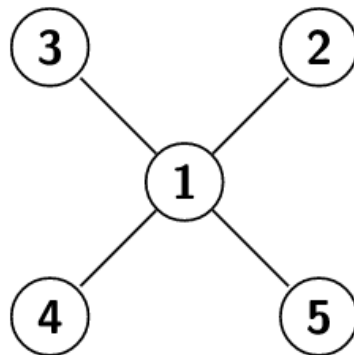
- the density of a graph of order $|V| = 1$ is undefined, both for algebraic reasons and, intuitively, because it can either be seen as perfectly sparse or perfectly dense
- all empty graphs have a density of 0 and are therefore sparse
- all complete graphs have a density of 1 and are therefore dense
- an undirected traceable graph has a density of at least $\frac{2}{|V|}$, so it is guaranteed to be dense for $|V| < 4$
- a directed traceable graph is never guaranteed to be dense
- a tournament has a density of $1/2$, regardless of its order

Exercise



Calculate the density of the graph and report if the graph is dense or sparse.

Exercise



This graph has a form $G(V, E)$, where $V = \{1, 2, 3, 4, 5\}$ and $E = \{(1, 2), (1, 3), (1, 4), (1, 5)\}$. Therefore, its first two characteristics are $|V| = 5$ and $|E| = 4$. Because the graph is undirected, we can calculate its maximum number of edges as:

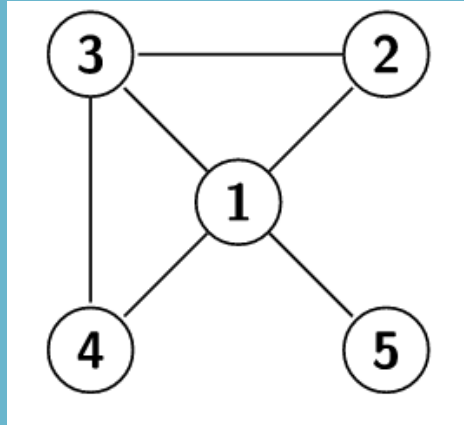
$$\text{Max}_U(V) = \binom{|V|}{2} = \frac{5 \cdot (5-1)}{2} = 10$$

From this, we can then calculate the density of the graph as:

$$D_U(V, E) = \frac{|E|}{\text{Max}_U(V)} = \frac{4}{10} = \frac{2}{5}$$

Because $D_U(V, E) < \frac{1}{2}$, we can conclude that $G(V, E)$ is a sparse graph.

Exercise

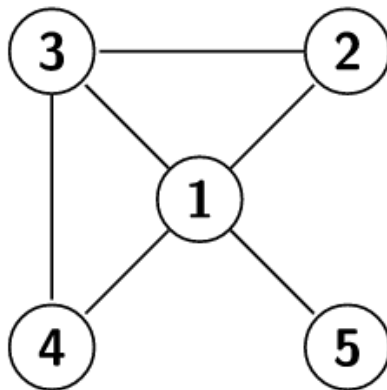


Here we added 2 edges

Again, calculate the density of the graph and see what change.

Exercise

If, instead, the graph had just two extra edges; say, $(2, 3)$ and $(3, 4)$, then it would look like this:



And the related calculations would change as follows:

$$|E| = 6 \rightarrow D_U(V, E) = \frac{|E|}{\text{Max}_U(V)} = \frac{6}{10} = \frac{3}{5}$$

This, in turn, makes the extended graph a dense graph, because $D_U > \frac{1}{2}$.

Efficiency in graph storage

The density of graphs affects the efficiency in the storage of the graph in memory. Graphs tend to be very large data structures, it is needed to store the graph in the format that is more efficient, in relation to its density.

Any graph can be expressed as a data structure in 3 manners:

- As a list of edges $E = [e_1, e_2, \dots, e_{|E|}]$
- Or, as an adjacency matrix $A_{|V|, |V|}$
- Or, adjacency lists $L_{|V|}$

These objects perform in a significantly different manner in terms of memory usage.

Exercise

- Let us suppose we have a sparse graph ($D=0.17$)
- $V=\{1,2,3,4,5,6\}$
- $E=\{(1,2),(2,3),(3,4),(4,5),(5,6)\}$
- How we represent the Edge List? And the Adj Matrix? And the Adj Lists?

Exercise

- Let us suppose we have a dense graph ($D=0.17$)
- $V=\{1,2,3,4,5,6\}$
- $E=(1,2),(1,3),(2,3),(2,4),(3,5),(4,5),(4,6),(5,6),(6,1),(3,1),(5,2),(6,4)\}$
- How we represent the Edge List? And the Adj Matrix? And the Adj Lists?



Efficiency in graph storage

As a general rule, the density of the graph dictates the preferable method for storage:

If the graph is **sparse**, we should store it as

- **Adjacency list** is generally preferred because it allows faster neighborhood queries (finding adjacent nodes).
- **Edge list** is simpler, but less efficient for operations like checking if an edge exists ($O(E)$ time complexity).

If the graph is **dense**, we should store it as an **adjacency matrix**

- Fast edge lookups ($O(1)$ time complexity)
- Higher memory usage ($O(|V|^2)$)



Thank you

Insert the title of your subtitle Here